T1: Problemas Clássicos de Concorrência

FPPD — Fundamentos de Processamento Paralelo e Distribuído (98713-04)

Objetivo

Aplicar os conceitos de programação concorrente estudados nas Unidades 1, 2 e 3, implementando em Go soluções para dois problemas clássicos de sincronização. As soluções devem utilizar goroutines, channels, `select` e os mecanismos de sincronização vistos em aula.

Além de implementar, o grupo deve **analisar** o comportamento concorrente de cada solução: onde há risco de deadlock, onde há risco de starvation e o que os dados de execução mostram sobre a distribuição de acesso aos recursos.

Resolver os **dois problemas** abaixo. Cada problema é entregue como um programa Go funcional e independente.

Problema 1: O Jantar dos Filósofos

Implementar o problema do Jantar dos Filósofos, proposto por Dijkstra, com `N` filósofos em uma mesa circular, utilizando channels para representar os garfos.

Requisitos:

- Implementar uma **versão base** que demonstre a ocorrência de deadlock (todos os filósofos pegam primeiro o garfo da esquerda).
- Implementar pelo menos **2 estratégias distintas** que eliminem o deadlock. Exemplos de estratégias discutidas em aula: quebra de simetria por hierarquia de recursos, limitação do número de filósofos simultaneamente à mesa, arbitragem centralizada por um processo garçom.
- Para cada versão (base e estratégias), apresentar uma **análise de deadlock e de starvation**: indicar se cada problema ocorre ou não e justificar com base nas condições de Coffman e nos conceitos vistos em aula.
- Coletar **dados de execução**: número de refeições de cada filósofo e tempo médio de espera pelos garfos ao longo de `R` iterações. Usar esses dados para argumentar sobre a **fairness** de cada estratégia.
- O programa deve receber `N` e `R` por parâmetro, e encerrar de forma ordenada ao final das iterações, sem goroutines bloqueadas.

Problema 2: Produtor/Consumidor com Buffer Limitado

Implementar o problema do produtor/consumidor com `P` produtores, `C` consumidores e um buffer compartilhado de capacidade `K`. Produtores bloqueiam quando o buffer está cheio; consumidores bloqueiam quando o buffer está vazio.

Requisitos:

- Implementar **duas versões** do buffer limitado:
  1. **Versão com channel:** o buffer é um channel com capacidade `K`, usando a semântica de bloqueio do próprio Go.
  2. **Versão com semáforos:** o buffer é uma estrutura de dados compartilhada, protegida explicitamente por semáforos de contagem (`notEmpty`, `notFull`) e por exclusão mútua nas

posições de escrita e leitura, reproduzindo o algoritmo clássico visto em aula. Com múltiplos produtores e múltiplos consumidores, não basta contar itens: é preciso impedir que dois produtores escrevam na mesma posição e que dois consumidores retirem o mesmo item.

- **Encerramento ordenado:** quando os produtores terminam, os consumidores devem drenar os itens restantes do buffer e encerrar sem deadlock e **sem perder nenhum item**. Ao final, o programa deve verificar que o total produzido é igual ao total consumido.
- **Consumo com timeout:** pelo menos um consumidor deve usar `select` com `time.After` para executar um comportamento alternativo quando nenhum item chega dentro de um intervalo definido.
- Coletar **dados de execução** variando a capacidade `K` do buffer (por exemplo, `K` igual a 1, 10 e 100) com `P` e `C` fixos: throughput total, itens consumidos por consumidor e ocupação média do buffer. Discutir o efeito de `K` sobre o desacoplamento entre produtores e consumidores.

Entrega

O grupo deve entregar **via Moodle, conforme os prazos definidos no cronograma da disciplina**, um arquivo `.zip` ou link para repositório Git (GitHub/GitLab) contendo:

1. **Código-fonte em Go**, organizado por problema (por exemplo, `filosofos/` e `produtor-consumidor/`), incluindo `go.mod` e instruções de compilação e execução.
2. **Relatório** (PDF, 3 a 5 páginas) contendo, para cada problema:
   - Breve descrição da solução adotada e das decisões de projeto.
   - **Análise de deadlock e starvation:** indicar se ocorrem ou não em cada versão e justificar com base nos conceitos vistos em aula.
   - **Análise de fairness** (Problema 1): tabela ou gráfico com os dados de execução coletados e discussão comparativa entre as estratégias.
   - **Análise do efeito do buffer** (Problema 2): dados coletados para os diferentes valores de `K` e discussão do comportamento observado, incluindo a comparação entre a versão com channel e a versão com semáforos.
   - Descrição de como o encerramento ordenado foi implementado em cada programa.

   O relatório deve incluir ainda uma seção sobre o **uso de ferramentas de IA**, conforme o item 4 das Regras Gerais.

Apresentações: conforme cronograma da disciplina no Moodle. Demonstrar a execução ao vivo, apresentar as análises e responder a perguntas sobre as decisões de projeto. A apresentação é **obrigatória e a participação é individual**: todos os integrantes devem estar presentes e ser capazes de responder a perguntas sobre qualquer parte do trabalho. **O integrante que não participar da apresentação não recebe nota no trabalho**, ainda que conste como autor do código ou do relatório. Impedimentos devem ser tratados com o professor antes da data da apresentação.

Grupos: até 4 integrantes, cadastrados como grupo no Moodle.

Critérios de Avaliação

O trabalho é avaliado com base nos seguintes critérios:

- Corretude das implementações: ausência de data race, nenhum item perdido, terminação ordenada.
- Análise de deadlock, starvation e fairness, sustentada pelos dados de execução coletados.
- Uso adequado de goroutines, channels, `select` e semáforos.
- Qualidade do relatório: clareza da descrição das soluções e das decisões de projeto.

- Apresentação e capacidade de responder a perguntas sobre a implementação.

Regras Gerais

1. **Linguagem:** Go.
2. **Bibliotecas externas:** permitidas para utilitários gerais. Não é permitido usar bibliotecas que abstraiam a concorrência do problema.
3. **Detecção de data race:** todos os programas devem executar sem erros com `go run -race`.
4. **Uso de IA:** o relatório deve conter uma seção **detalhando o uso de ferramentas de IA** (ChatGPT, Claude, Copilot e similares): quais ferramentas foram utilizadas, em que etapas do trabalho (projeto da solução, geração de código, depuração, análise dos dados, redação do relatório) e de que maneira. Caso nenhuma ferramenta tenha sido utilizada, declarar isso explicitamente. Independentemente do uso, cada integrante deve ser capaz de explicar qualquer parte do código e das análises entregues.
5. **Apresentação obrigatória:** a apresentação é parte indissociável da avaliação. O integrante que não participar da apresentação **não recebe nota no trabalho**, independentemente da sua contribuição ao código ou ao relatório.
6. **Plágio e referências:** soluções copiadas da internet ou de outros grupos resultam em nota zero. Consultar material externo é permitido e esperado, desde que **todas as fontes consultadas sejam referenciadas no relatório**: livros, artigos, documentação, repositórios, vídeos e páginas web que tenham influenciado a solução. Deixar de referenciar uma fonte efetivamente utilizada é tratado como plágio. Cada integrante do grupo deve ser capaz de explicar qualquer decisão de projeto durante a apresentação.
7. **Formato de entrega:** conforme a seção “Entrega” acima.